

LAB 1: Digital Sampling, Fourier Filtering, and Heterodyne Mixers in Radio Astronomy

Contents

1	Introduction	2
2	Goals	2
3	Schedule	3
4	Software Engineering	3
5	Digitally Sampling a Sine Wave (Lab Activity, Week 1)	3
5.1	Handouts and Software	3
5.1.1	The ugradio Python Package	3
5.2	Optional: Listen to FM	4
5.3	Digital Sampling and the Nyquist Criterion	4
5.3.1	Sampling with an SDR and a Raspberry Pi	4
5.4	Voltage Spectra and Power Spectra	5
5.5	Leakage Power	5
5.6	Frequency Resolution	5
5.7	Power Spectra in Other Nyquist Windows	5
5.8	Fourier Transforms of Noise	6
5.8.1	Use the Laboratory Noise Generator	6
6	Fourier Transforms, Analytic and Discrete (At Home, Week 1)	6
6.1	The Analytic Fourier Transform	6
6.2	The Discrete Fourier Transform (DFT)	7
6.3	Power Spectra and Discrete Fourier Transforms	7
6.4	The Power Spectrum and the Autocorrelation Function (ACF)	7
7	Mixers (Lab Activity, Week 2)	8
7.1	The Double-SideBand (DSB) Mixer	8
7.2	Intermodulation Products in Real Mixers	9
7.3	The Single-Sideband Mixer (SSB Mixer)	9
7.3.1	Reverting to a DSB Mixer	9
7.3.2	The SSB Mixer	9
7.3.3	The R820T chip	9
8	On Mixers and the Heterodyne Process (At Home, Week 2)	10
8.1	The Heterodyne Process	10
8.2	Single Sideband (SSB) Mixer Theory	10
8.3	Double Sideband (DSB) Mixer Theory	10
9	Writing the Lab Report (Week 3)	12

1. Introduction

This lab explores digital samplers, discrete Fourier Transforms, and mixers. These are tools at the core of radio astronomy and technologies like cell phones and WiFi (which was invented by a radio astronomer).

These experiments introduce practical radio astronomy. You will configure lab equipment, collect and analyze data, and present findings. Avoid **technical debt** by investing *now* in your understanding and code base; everything you build will be crucial for Lab 2, where you will observe hydrogen’s 21-cm line emission in the Milky Way.

Success requires good research practices. Your grade depends on the report due in three weeks. While you need not complete every task, your report must thoroughly address the goals in §2.

To enhance your lab experience and report:

- Maintain a **lab notebook**. Detailed notes are vital for interpreting data and troubleshooting. We cannot help you if you forget your experimental configuration.
- **Version control** your data, code, and plots. This safeguards against data loss and facilitates reproducibility, particularly when paired with code generators.
- Begin your report early. Plan out the data and plots needed for your report to optimize lab time. Write incrementally to avoid facing a blank page at the deadline.
- Use lectures, AstroBaki, datasheets, and these instructions. Real lab work requires finding trustworthy resources. Take notes.
- Write in the style of scientific publications. Guide the reader to trust your results. Focus on synthesizing findings rather than chronicling the process. Ensure all text and plots are original and your *individual* work. Cite external sources. Follow our AI policy regarding the use of generative AI in your work.

We evaluate your report on your theoretical background, experimental setup, data analysis, and quantitative interpretation.

Your report will also implicitly showcase your proficiency in these technical skills (no explicit discussion required):

- Configuring lab equipment and operating Linux on Raspberry Pi.
- Using Python for experimental control, data acquisition, analysis, signal processing, and plotting.
- Writing a technical report in L^AT_EX (or for Lab 1, Jupyter markdown), including figures and captions.

2. Goals

Your report should address the following main objectives:

1. Demonstrate an understanding of Nyquist sampling and aliasing.
2. Characterize the bandpass filter of the SDR.
3. Use and accurately describe power and voltage spectra.
4. Investigate spectral leakage and frequency resolution.
5. Analyze the properties of noise and the radiometer equation.
6. Demonstrate the operation of DSB and SSB mixers and explain their importance.
7. Accurately present quantitative results in the context of expectations, theory, tests, and diagnostics that demonstrate the correctness of the analysis.

8. Build an installable Python package, with revision control and unit tests, that supports your report. Provide the link in your report.

3. Schedule

This lab is ambitious and has a steep learning curve. Do not fall behind. Students consistently underestimate the time required to write a report.

1. *Week 1:* Complete §5 and review §6. Be ready to demonstrate your work, software, and results in class.
2. *Week 2:* Finish §7 and review §8. Prepare to present your findings and progress.
3. *Week 3:* Consult §9 for guidance on writing and figures. Craft a clean Jupyter notebook that addresses the objectives in §2.

4. Software Engineering

Programming complexity increases throughout the semester. Develop good practices in organizing, documenting, testing, and revisioning your code. Software engineering is a vital (and marketable) skill. Start with these principles:

- *Package your code:* Move beyond Jupyter notebooks. Build infrastructure code (functions and classes) into **modules** and **packages**. This enhances organization, testability, and reusability. See <https://docs.python.org/3/tutorial/modules.html>.
- *Code (at least) twice:* Write code once to understand the problem. Recode it for organization and structure. Thoughtful interface design limits complex dependencies and speeds debugging.
- *Test your code:* How do you know your code works? You can only trust code you have tested—this is only becoming more true with generative AI. Use modular **unit testing** for large projects. Explore Python’s **unittest** or *pytest* modules.
- *Control your revisions:* Use **git** to track changes. A GitHub account facilitates backup, synchronization, and collaboration. The learning curve is worth the effort.

Professional software engineering uses online repositories and continuous integration. While we will not use these advanced methods, understanding them is beneficial.

Separate data acquisition (lab control) from data analysis. Use command line scripts for acquisition and Jupyter notebooks for analysis. Create shared modules importable by both.

5. Digitally Sampling a Sine Wave (Lab Activity, Week 1)

5.1. Handouts and Software

In week 1, explore digital sampling and spectral analysis using Linux and Python. Refer to Astrobaki¹ to review *Nyquist Sampling* and *The Fourier Transform*. Review primers on Linux/Unix, Python, Text Editors, and Revision Control.

5.1.1. The `ugradio` Python Package

The `ugradio` package supports these labs. It is pre-installed on your Raspberry Pi. Verify it by running `import ugradio` in IPython or Jupyter. Focus on these modules:

- `ugradio.sdr` — Interfaces with the Software-Defined Radio (SDR).

¹https://casper.astro.berkeley.edu/astrobaki/index.php/Undergraduate_Radio_Lab

- `ugradio.dft` — Computes arbitrary Discrete Fourier Transforms. It is much slower than `numpy.fft` but more flexible.

Explore the source code on GitHub, in `ugradio/ugradio_code/src`, or via the `?` or `??` commands in Jupyter/IPython.

5.2. Optional: Listen to FM

You may use `gqrx` to listen to FM radio. Set the `Device String` to:

```
default_input=True,default_output=True,device_id=0,driver=audio
```

Press `Play` and adjust settings (WFM Stereo, Hardware AGC). US FM radio spans 87.8–108 MHz.

FM broadcasts are entertaining, but we must understand the signal path.

5.3. Digital Sampling and the Nyquist Criterion

Explore the Nyquist criterion using an SDR and Python simulations. Vary the signal frequency (ν_0) and sampling frequency (ν_s). Compare your results with theory. Your report must analyze aliasing by integrating theory, data, and simulations. Pose and quantitatively answer a specific question.

- “*What is aliasing?*” is too broad.
- “*How many Nyquist zones can I demonstrate?*” is better.
- “*Can I characterize the SDR bandpass filter using purposeful aliasing?*” is better still.

5.3.1. Sampling with an SDR and a Raspberry Pi

Use a prepared SD card or follow the AstroBaki setup instructions. Connect an SDR (e.g., Nooelec NESDR SMART) to the blue USB 3.0 port. Regularly update the `ugradio` package:

1. `cd ~/ugradio/ugradio_code`
2. `git pull`
3. `pip install .`

To start, bypass the tuner and use the RTL2832U ADC directly. This is the **direct sampling** mode described in `ugradio.sdr`. Use a signal generator to produce a < 500 kHz sine wave with < 3 mV peak-to-peak voltage. Verify the signal with an oscilloscope before connecting the SDR. Our course mantra is: *mistrust everything!* Make each component prove to you that it is working.

Write Python code to create a `ugradio.sdr.SDR` object. Capture data at varying sample rates ($\nu_s = 1.0$ – 3.2 MHz). Plot the data to ensure it looks sinusoidal. *Inspect your data carefully; the first capture may contain stale buffers.* Observe how the amplitude changes with frequency due to internal anti-aliasing filters. You can override these filters to illustrate aliasing:²
`fir_coeffs=np.array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2047])`.

Investigate Nyquist zones and filter responses by varying ν . Document your methods and settings. Compare amplitudes with default and aliasing FIR coefficient settings.

Write scripts to save data using `numpy.savez`. Load files in Jupyter with `numpy.load` for interactive plotting. Compute power spectra using `numpy.fft` or `ugradio.dft` (see §6.2 and §6.3). Mind your data order (e.g., `numpy.fft.fftshift`) and units.

Convert your best notebook plots into revision-controlled scripts. Label all axes with appropriate units. Restrict ranges to clearly illustrate signal shape and frequency. Write self-contained

²I expanded (hacked) the `librtssdr` driver to support FIR filter coefficients. FIR filters implement convolutions, discussed next week.

captions—usually three sentences at minimum—for every plot destined for your report.

Synthesize several data runs into a single plot to illustrate Nyquist’s criterion. You will produce many plots while exploring your data, but your report should only include those that quantitatively illustrate specific points. One good plot is much better than ten vague ones.

5.4. Voltage Spectra and Power Spectra

Produce power and voltage spectra and distinguish their respective roles. Power spectra can be integrated to summarize frequency content. Voltage spectra provide instantaneous phase information via real and imaginary components. When is one preferred over the other?

Consider the complex voltage spectrum. What do the real and imaginary parts represent? How are negative frequencies realized? Explore these concepts in your report.

Plot the real (`spec.real`) and imaginary (`spec.imag`) components. Observe symmetries between negative and positive frequencies. Verify consistency with independent data. Explain amplitude and sign variations.

Use `dft.idft` or `numpy.fft.ifft` to perform the inverse Fourier Transform of a *power* spectrum. Ensure correct data ordering for correct frequency inference. Compute the autocorrelation function (ACF) from the voltage time series using `numpy.correlate` and `scipy.signal.correlate`.

The correlation theorem states the Fourier transform of the power spectrum matches the ACF. Compare them (and explore padding your data with zeros) to find similarities and discrepancies. Explain them.

5.5. Leakage Power

The `dft` by default calculates a Fourier spectrum at N frequencies for N samples, for a frequency separation of $\Delta\nu = \nu_s/N$. FFT operations hard-code this spacing.

Calculate the power spectrum for waveforms where $N_{\text{freq}} \gg N$. Use `dft` with $\Delta\nu \ll \nu_s/N$. Use a logarithmic vertical axis to find power at frequencies other than ν_0 . Power at $\nu \neq \nu_0$ indicates **spectral leakage**.

Use the Convolution Theorem to mathematically explain why you find power at $\nu \neq \nu_0$ with a Discrete Fourier Transform.

5.6. Frequency Resolution

Determine the minimum distinguishable spacing between two spectral lines.

Combine two function generators using a power splitter in reverse. Select two close frequencies and plot their power spectrum. Use dense frequency spacing.

Experimentally determine how the number of samples used in a Fourier Transform affects the **frequency resolution**. Provide a mathematical explanation for your findings using the Convolution Theorem.

5.7. Power Spectra in Other Nyquist Windows

Calculate power spectra beyond the standard $\pm\nu_s/2$ range.

Use a Nyquist-sampled time series and calculate spectra over $\pm W\nu_s/2$ (where $W \geq 4$). Each W represents a different **Nyquist window**. Compare the spectra across windows. For $W > 1$, spectra include frequencies that seemingly violate the Nyquist criterion but remain interpretable. Lab 4 uses the 12th Nyquist window; why does this work?

This illustrates that the signal’s *bandwidth* (inclusive of negative frequencies) must not exceed

ν_s , rather than the absolute frequency. What happens if the bandwidth violates this?

5.8. Fourier Transforms of Noise

Scientifically, noise refers to random fluctuations from a known statistical distribution (typically Gaussian). Use the term carefully in your report, distinguishing it from systematic errors or bias.

Many astronomical signals appear as noise in the electric field. We usually measure the power ($P \propto V^2$) of that noise. Blackbodies emit waves described by the Planck function:

$$B_\nu = \frac{2h\nu^3}{c^2} \frac{1}{e^{h\nu/kT} - 1}. \quad (1)$$

In the Rayleigh-Jeans limit ($h\nu/kT \ll 1$):

$$B_\nu \approx \frac{2kT}{\lambda^2}. \quad (2)$$

Noise power depends linearly on temperature T .

Radio astronomers express noise power as brightness temperature T_B , defined as $I \equiv 2kT_B/\lambda^2$, even for non-thermal sources like synchrotron emission.

All power sources, including receiver noise, relate to a temperature. Receiver noise (e.g. Johnson noise from thermal electrons) is typically Gaussian (Central Limit Theorem) and characterized by receiver temperature T_{rx} . This adds to the source temperature T_B to yield the total temperature as a function of frequency.

5.8.1. Use the Laboratory Noise Generator

The lab noise generator uses a diode to produce broadband random voltages. Use a band-pass filter to prevent aliasing before direct sampling.

- Capture tens of thousands of samples. Analyze their mean and variance.
- Histogram the samples using `numpy.histogram`. Overplot a theoretical Gaussian with width equal to the rms voltage. Explain differences.
- Collect multiple data blocks. Compute the power spectrum for each and the average across all.
- Compare a single block's spectrum to the average of N blocks ($N = 2, 4, 8, 16$). How does time averaging affect signal-to-noise ratio (SNR)? Does SNR scale as N^x for some x ? Compare with Central Limit Theorem predictions.
- Calculate the ACF for a 16,000 sample block. Compare the power spectrum derived from the ACF with the Fourier transform result. Compare the FWHM of the ACF ($\Delta\tau_{FWHM}$) with the FWHM of the power spectrum (ΔF_{FWHM}). Are they related?

6. Fourier Transforms, Analytic and Discrete (At Home, Week 1)

6.1. The Analytic Fourier Transform

A Fourier transform maps a function between complementary coordinates. The input is usually a time-domain signal $E(t)$; the output is in frequency domain:

$$\tilde{E}(\nu) = \int_{-T/2}^{T/2} E(t) e^{-2\pi i \nu t} dt. \quad (3)$$

Because of the imaginary exponent, $\tilde{E}(\nu)$ is complex-valued. The Fourier Transform is **invertible**:

$$E(t) = \frac{1}{B} \int_{-B/2}^{B/2} \tilde{E}(\nu) e^{2\pi i \nu t} d\nu . \quad (4)$$

This works because complex sine waves form a **basis** for the space of functions. Any function is expressed exactly by its Fourier coefficients.³

The Fourier transform is a linear operator (like rotation) with these properties:

1. Linearity: FT of a sum is the sum of FTs.
2. Invertibility: No information is lost.
3. Unitarity: Power is preserved.

6.2. The Discrete Fourier Transform (DFT)

Discrete Fourier transforms differ from analytic ones because sampled signals are discontinuous. Integrals become simple sums. The `ugradio.dft` module allows manual frequency specification. It is slow but transparent to understand. Use it when you need to oversample output frequencies.

Recommendations for `ugradio.dft`:

- Use even N so the zero frequency falls in a bin center.
- Center time $t = 0$. For even N , run times from $-\frac{N}{2}/\nu_s$ to $(\frac{N}{2} - 1)/\nu_s$.
- Calculate output for N frequencies from $-\frac{\nu_s}{2}$ to $+\frac{\nu_s}{2} (1 - \frac{2}{N})$. This makes $\Delta\nu = \nu_s/N$.

Use `ugradio.dft.<tab>` or `ugradio.dft??` to explore the module.

In practice, given the Nyquist criterion and fixed bandwidth of each channel, algorithmic shortcuts allow DFTs to be computed extremely efficiently. `numpy.fft` is fast but (even over millions of samples), but assumes equally spaced input time samples and output frequencies. It also orders output frequencies strangely: 0 to positive max, then negative max to 0. `numpy.fft.ifft` requires this order as input.

The function `fftshift` reorders frequencies for plotting (negative to positive). **Only use `fftshift` for plotting**. It breaks `ifft`. Use `fftfreq` to calculate output frequencies.

6.3. Power Spectra and Discrete Fourier Transforms

The **power spectrum** P_ν is proportional to voltage squared. For complex quantities, multiply the voltage by its conjugate:

$$P_\nu = \tilde{E}(\nu) \tilde{E}^*(\nu) . \quad (5)$$

In Python: `P = E * E.conj()`. The imaginary part should be zero. If not, cast as float to remove rounding errors.

Alternatively, use `P = numpy.abs(E)**2`. This automatically returns a real result.

6.4. The Power Spectrum and the Autocorrelation Function (ACF)

Radio astronomy requires facility with the **convolution** and **correlation** theorems.

Convolution of E and F :

$$[E * F](\tau) = \int_{-T/2}^{+T/2} E(t) F(\tau - t) dt, \quad (6)$$

³Analytically, limits B and T are infinite. In practice, they are bounded, causing spectral leakage. Note: our forward FT does not divide by the interval, but the inverse does. This matches `numpy.fft`.

Correlation:

$$[E \star F](\tau) = \int_{-T/2}^{+T/2} E(t)F(\tau + t) dt. \quad (7)$$

τ represents the time shift (delay) between functions. Convolution flips the sign of t in F ; correlation does not.

The Convolution Theorem:

$$[\widetilde{E * F}](\nu) = \tilde{E}(\nu) \cdot \tilde{F}(\nu) \quad (8)$$

The Correlation Theorem:

$$[\widetilde{E \star F}](\nu) = \tilde{E}(\nu) \cdot \tilde{F}^*(\nu) \quad (9)$$

The FT of the convolution/correlation is the product of the FTs.

When $E(t) = F(t)$, correlation becomes the **Autocorrelation function** (ACF). Equation 9 states the power spectrum is the Fourier transform of the ACF.

These theorems assume $T \rightarrow \infty$. Digital ACFs suffer from end effects. For N samples with delay ΔN , the number of summed terms decreases as ΔN increases.

7. Mixers (Lab Activity, Week 2)

7.1. The Double-SideBand (DSB) Mixer

Build a double sideband (DSB) mixer (Figure 1). A mixer multiplies two input voltage signals. The Radio Frequency (RF) enters one port, the Local Oscillator (LO) another, and the Intermediate Frequency (IF) exits the third.

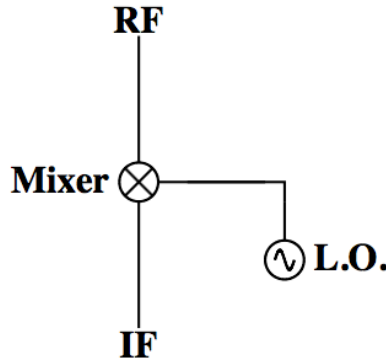


Fig. 1.— A DSB mixer. The RF input is the ‘signal’, LO is the local oscillator, and IF is the product

We use Mini-Circuits ZAD-1 and ZFM-15 mixers. Ports are labeled R (RF), L (LO), and X or I (IF). R and L ports do not couple to DC; check datasheets for frequency ranges.

Use a mixer to build a DSB. Assign one signal generator as LO (ν_{LO}) and another as RF ($\nu_{RF} = \nu_{LO} \pm \Delta\nu$). Set $\Delta\nu$ small ($\sim 5\%$) compared to ν_{LO} . Set input power to 0 dBm. The output contains sum and difference frequencies.

Digitally sample the output. Identify sum and difference frequencies. Choose a sample rate that satisfies Nyquist and captures the wave shape. Ensure you sample enough points to capture the slowest sine wave period.

Plot power spectra for $\nu_{RF} = \nu_{LO} + \Delta\nu$ and $\nu_{RF} = \nu_{LO} - \nu$. Explain the plots using the terms upper sideband and lower sideband.

Plot the waveform for one case. Filter the sum frequency by zeroing it in the Fourier domain (**Fourier filtering**). Inverse transform and plot the filtered signal. Explain the result.

7.2. Intermodulation Products in Real Mixers

Real mixers use nonlinear diodes, not ideal multipliers. This produces harmonics called **intermodulation products**. Proper signal levels minimize these, but they can still affect measurements.

Examine a power spectrum with a logarithmic axis. Identify the "forest" of lines. Trace strong lines to harmonics of the main signals.

7.3. The Single-Sideband Mixer (SSB Mixer)

An SSB mixer (Figure 2) uses two DSB mixers. The right-hand LO is phase-shifted 90° . One output mixes with a cosine, the other with a sine. This produces real (I) and imaginary (Q) components ("IQ sampling"). The power spectrum distinguishes positive and negative frequencies. Connect I and Q to an oscilloscope.

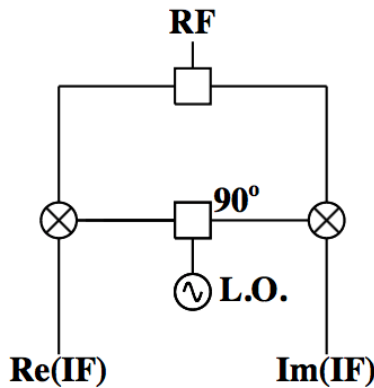


Fig. 2.— An SSB mixer. The important part is the 90° phase delay in the right-hand LO.

Following figure 2, construct an SSB mixer using a cable for phase delay. Use a short cable for no delay and a long cable for 90° (or 450°) delay ⁴.

7.3.1. Reverting to a DSB Mixer

Use a short cable (near zero delay). The two halves are identical. Take time series data for upper and lower sidebands. Compare the phase of I and Q. Do you see a difference?

7.3.2. The SSB Mixer

Use the 90° delay cable. Can you distinguish positive and negative $\Delta\nu$ in the I/Q phase? Why?

7.3.3. The R820T chip

The SDR has a built-in mixer. Tune the LO in software (turn off **direct** sampling). Sample I and Q data into a complex array. Calculate power spectra. Can you distinguish positive and negative $\Delta\nu$? Compare the internal mixer waveforms to your external mixer results.

⁴In vacuum, light travels about 1 foot per ns—the only legitimate use of imperial units in astronomy. In a cable, light travels about 70% slower than in vacuum.

8. On Mixers and the Heterodyne Process (At Home, Week 2)

8.1. The Heterodyne Process

Mixers shift the entire input spectrum by a uniform amount. They multiply the signal by a Local Oscillator (LO) sine wave (ν_{LO}).

This allows antennas to tune to incoming radiation while filters and samplers operate at lower fixed frequencies. AM radios, for example, shift MHz signals to audio (kHz) frequencies. Tuning a station changes the LO frequency; the chain remains static.

This is a **heterodyne** receiver.

8.2. Single Sideband (SSB) Mixer Theory

SSB theory is best understood with complex numbers and **negative frequencies**. Euler's formula defines a complex sinusoid:

$$Ae^{i\omega t} = A \cos(\omega t) + i \cdot A \sin(\omega t). \quad (10)$$

Negative frequencies might seem weird. How can something oscillate a negative number of times per second? Negating frequency swaps the sign of the imaginary component. Negative frequency implies a phase relationship between sine and cosine.

Consider an idealized SSB mixer with LO $e^{-i\omega_0 t}$. Input signal:

$$E(t) = A \sin(\omega_0 - \Delta\omega)t + B \sin(\omega_0 + \Delta\omega)t. \quad (11)$$

Mixing multiplies $E(t)$ by the LO. Using Euler's formula, the product contains beat frequencies $e^{\pm i\Delta\omega t}$ and high frequency terms ($2\omega_0$). Low-pass filtering removes the high frequencies:

$$\text{LPF} [E(t) \cdot e^{-i\omega_0 t}] = \frac{A}{2i} e^{-i\Delta\omega t} + \frac{B}{2i} e^{i\Delta\omega t}. \quad (12)$$

The signals shift to lower frequencies. Retaining real and imaginary parts preserves the distinction between A and B . Positive and negative **sidebands** remain separate.

Figure 2 shows the SSB mixer. The left mixer is DSB. The right mixer uses a 90° delayed LO. Its output ($\text{Im}(\text{IF})$) becomes the sine component to the left's cosine ($\text{Re}(\text{IF})$).

8.3. Double Sideband (DSB) Mixer Theory

DSB mixers are simpler to build but harder to interpret. Using LO $\sin \omega_0 t$ and signal $E(t) = A \sin(\omega_0 + \Delta\omega)t$:

$$E(t) \cdot \sin \omega_0 t = A [\cos \Delta\omega t - \cos(2\omega_0 + \Delta)t]. \quad (13)$$

Filtering yields a beat frequency.

However, if $E(t) = A \sin(\omega_0 + \Delta\omega)t + B \sin(\omega_0 - \Delta\omega)t$:

$$\text{LPF} [E(t) \cdot \sin \omega_0 t] = (A + B) \cos \Delta\omega t. \quad (14)$$

Both $\omega_0 + \Delta\omega$ and $\omega_0 - \Delta\omega$ map to the same output frequency. The DSB mixer cannot distinguish positive and negative deviations. It stacks sidebands on top of each other.

Figure 3 illustrates this. DSB mixing creates symmetric spectra for USB and LSB. Without filters, they sum inextricably. SSB mixing retains separation.

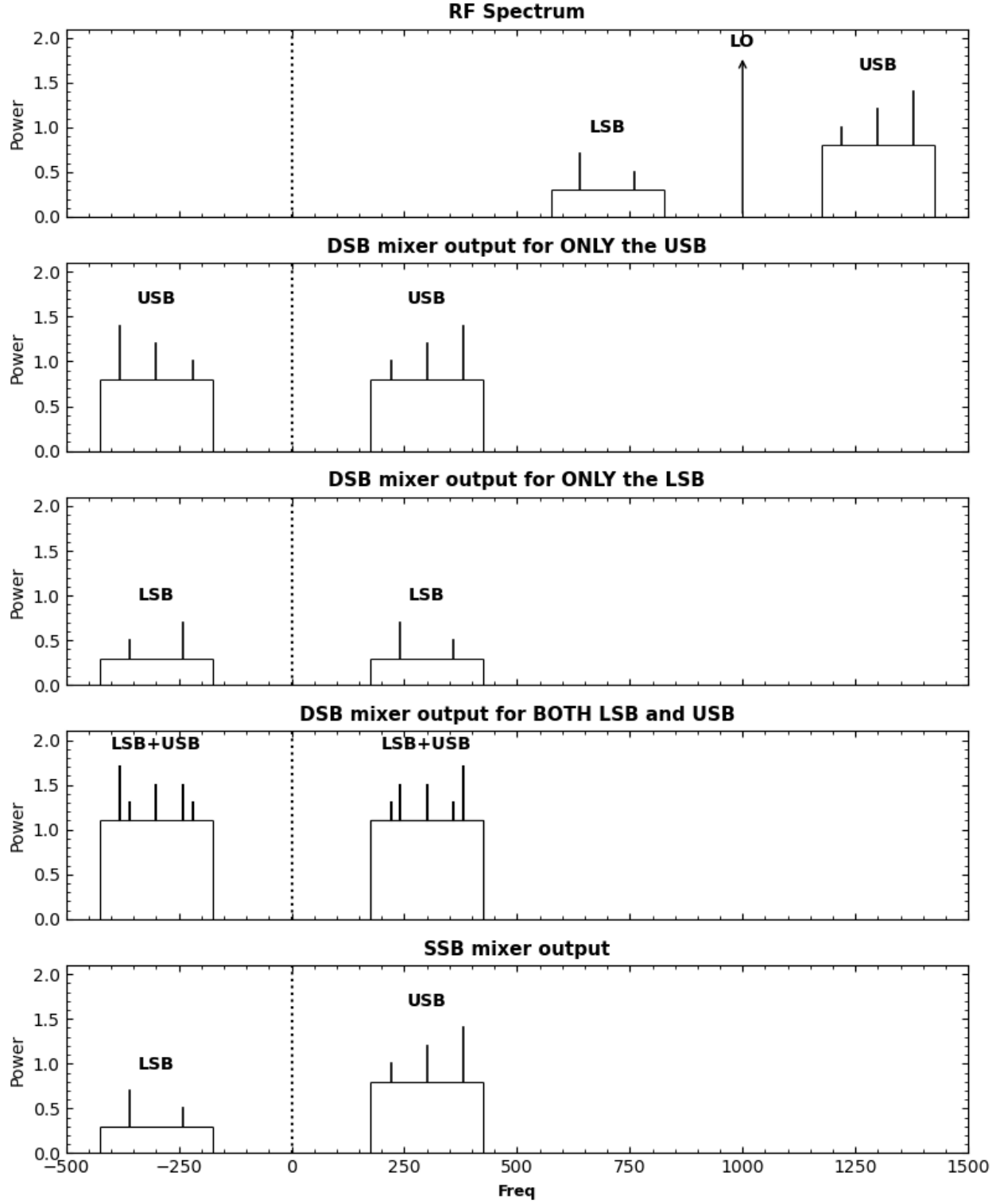


Fig. 3.— Upper (USB) and lower (LSB) sidebands in DSB and SSB mixers. Panel 1: RF spectrum. Panel 2/3: USB/LSB after DSB mixing (symmetric). Panel 4: DSB sum (indistinguishable). Panel 5: SSB separation.

9. Writing the Lab Report (Week 3)

For the first lab **only**, your submission will be a Jupyter notebook consisting of code, equations, detailed explanations, high-quality plots, and detailed captions. Review the report guidelines on AstroBaki. *Your notebook must use functions/classes created by you in a publicly viewable, revision controlled, installable package hosted on GitHub.* This package should also include any scripts used to take data. Identify the url of your package and include the installation command at the top of your notebook.

The notebook you turn in should be a polished product. All cells must run sequentially, with final plots produced inline. Provide enough description for a peer to understand (and reproduce) your results. Choose axis limits and scales carefully. Don't include old/broken code, pages of numerical output, or anything else that makes your notebook hard to read. As discussed in class, all code, write-ups, and analysis must be your own.

Address the goals in §2 directly. Focus on conveying high-level results, not a laundry list of tasks you performed. Document process only as necessary to support your results.

Writing takes time. Start early.